Informe Laboratorio Semana 9 — De n-capas a DIP y DI

Arquitectura de Sistemas I — 2026-2

Integrantes	Juan Rojas (jdrojasp11) Adrián Camaño
Repositorio (fork)	https://github.com/jdrojasp11/demo-cliente-servidor
Rama principal	`main` (versión n-capas Semana 8, sin modificar)
Rama de trabajo	`refactor-dip-di` (DIP + DI + pruebas)
Repositorio de referencia	ebuitrago/demo-cliente-servidor (upstream, no modificado)
Fecha	2026-10-21

1. Objetivo

Refactorizar la versión en capas de la Semana 8 aplicando el **Principio de Inversión de Dependencias (DIP)** y la **Inyección de Dependencias (DI)**, de modo que la capa de aplicación deje de importar módulos concretos de persistencia. El criterio de éxito es poder ejecutar los casos de uso con repositorios simulados y conservar los endpoints existentes con PostgreSQL.

2. Preparación y rama

```
git clone --branch main https://github.com/ebuitrago/demo-cliente-servidor.git demo-citas-semana9-local
git switch -c refactor-dip-di
npm ci
New-Item -ItemType Directory -Force -Path "src/aplicacion/contratos", "test"
```

Estado de la rama antes de crear: `main`. Después: `refactor-dip-di`. `npm ci` instaló 85 paquetes sin modificar el bloqueo.

3. Protección de credenciales (.gitignore)

```
node_modules/
.env
```

Comando	Resultado

`git check-ignore -v --no-index .env`	`.gitignore:2:.env  .env` **(regla activa)**
`git ls-files -- .env`	(sin salida) **no está bajo seguimiento**

4. Cambios aplicados (DIP + DI)

4.1 Contrato estructural — `src/aplicacion/contratos/repositorios.js` (nuevo)

```
function exigirMetodos(objeto, metodos, nombre) {
  for (const metodo of metodos) {
    if (typeof objeto?.[metodo] !== 'function') {
      throw new TypeError(`${nombre} debe implementar ${metodo}()`);
    }
  }
}
function verificarRepositorios({ citaRepository, profesionalRepository }) {
  exigirMetodos(citaRepository,
    ['listarTodas', 'existeEnHorario', 'guardar'], 'citaRepository');
  exigirMetodos(profesionalRepository, ['listarTodos'], 'profesionalRepository');
}
module.exports = { verificarRepositorios };
```

4.2 Servicio de aplicación — `src/aplicacion/citaService.js`

Se eliminaron los `require('../persistencia/...')`. El servicio ahora es una **factoría** que recibe los repositorios y un **reloj inyectado** para pruebas reproducibles.

```
const reglas = require('../dominio/reglasDeAgenda');
const { verificarRepositorios } = require('./contratos/repositorios');

function crearCitaService({ citaRepository, profesionalRepository,
                            reloj = () => new Date() }) {
  verificarRepositorios({ citaRepository, profesionalRepository });
  return {
    consultarCitas: () => citaRepository.listarTodas(),
    consultarProfesionales: () => profesionalRepository.listarTodos(),
    reservarCita: async (datos) => {
      reglas.validarDatosCompletos(datos);
      reglas.validarFechaFutura(datos.fecha_hora, reloj());
      const ocupado = await citaRepository.existeEnHorario(
        datos.profesional_id, datos.fecha_hora);
      reglas.validarAgendaLibre(ocupado);
      const id = await citaRepository.guardar(datos);
      return { mensaje: 'Cita creada', id };
    }
  };
}
module.exports = { crearCitaService };
```

4.3 Rutas Express — `src/presentacion/citasRoutes.js`

La factoría recibe el servicio ya construido y conserva los cuatro endpoints.

```
function crearCitasRoutes({ citaService }) {
  const router = express.Router();
  const CODIGO_HTTP = { DATOS_FALTANTES: 400, FECHA_INVALIDA: 400,
                        FECHA_PASADA: 400, AGENDA_OCUPADA: 409 };
  router.get('/salud', ...);
  router.get('/citas', ...);
  router.get('/profesionales', ...);
  router.post('/citas', ...);
  return router;
}
module.exports = { crearCitasRoutes };
```

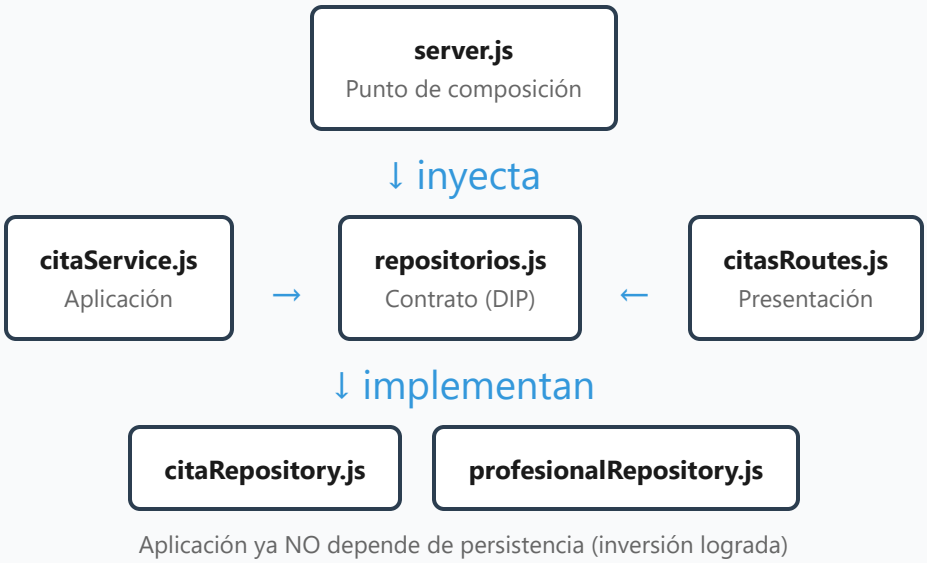
4.4 Punto de composición — `server.js`

Único lugar donde se importan implementaciones concretas y se inyectan.

```
const { crearCitaService } = require('./src/aplicacion/citaService');
const citaRepository = require('./src/persistencia/citaRepository');
const profesionalRepository = require('./src/persistencia/profesionalRepository');

const citaService = crearCitaService({ citaRepository, profesionalRepository });
app.use('/api', crearCitasRoutes({ citaService }));
```

5. Diagrama de dependencias



Antes (Semana 8): `citaService.js` → `require('../persistencia/...')`.
Después (Semana 9): `server.js` → repositorios → servicio → rutas; el servicio depende de abstracciones.

6. Comprobación de sintaxis y desacoplamiento

```
node --check src/aplicacion/contratos/repositorios.js   # OK
node --check src/aplicacion/citaService.js              # OK
node --check src/presentacion/citasRoutes.js            # OK
node --check server.js                                  # OK
node -e "require('./src/presentacion/citasRoutes')..."  # function
Get-ChildItem .\src\aplicacion -Filter *.js -Recurse |
  Select-String -Pattern 'persistencia/'                 # sin resultados
```

La capa de aplicación no contiene ninguna importación hacia módulos concretos de persistencia.

7. Pruebas unitarias (sin Supabase, sin Internet)

```
node --test test/citaService.test.js
```

```
✓ reserva permitida
✓ agenda ocupada no genera escritura
✓ fecha pasada se rechaza sin consultar PostgreSQL
✓ Rechaza repositorios que no cumplen el contrato
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

Usan `node:test` y repositorios en memoria. No inician Express, no requieren `.env` y no escriben en PostgreSQL.

8. Pruebas HTTP locales

Para validar los casos 201 y 409 se levantó una instancia **PostgreSQL 17.6 local** (puerto 5433, SSL activo) con los binarios portables, se aplicó `db/setup.sql` y se apuntó `DATABASE_URL` a esa base propia de pruebas. No se usó la base de producción de Render.

Petición	Resultado	Observación
`GET /api/salud`	**200**	`{"estado":"ok","servidor":"activo",...}`

`GET /api/profesionales`	**200**	3 profesionales desde PostgreSQL
`GET /api/citas`	**200**	lista de citas
`POST /api/citas` (futura)	**201**	Cita creada, devuelve `id`
`POST /api/citas` (repetida)	**409**	AGENDA_OCUPADA
`POST /api/citas` (fecha 2020)	**400**	FECHA_PASADA (validación de dominio, sin escritura)

```
node test/api-smoke.js
API comprobada: GET 200, POST 201, 409 y 400.
```

Las cuatro pruebas unitarias (4/4) y el smoke test HTTP completo (200/201/409/400) quedan aprobados.

9. Publicación de la rama

```
git add .gitignore server.js src test
git diff --cached --name-only   # .env y node_modules NO incluidos
git commit -m "Semana 9: aplicar DIP y DI y ejecutar pruebas"
git push --force-with-lease origin refactor-dip-di
```

La rama se publicó únicamente en el fork del equipo (`jdrojasp11/demo-cliente-servidor`). No se hizo merge a `main`, no se creó pull request y no se desplegó en Render. El repositorio del docente (`upstream`) permanece intacto.

10. Conclusiones

- La capa de aplicación pasó de depender de módulos concretos a depender de un **contrato** (DIP).
- Las dependencias se construyen e inyectan en `server.js`, el punto de composición (DI).
- El reloj inyectado permite pruebas deterministas de la regla de fecha futura.
- Las 4 pruebas unitarias se ejecutan sin base de datos ni red.
- Los endpoints de la API se conservan sin cambios funcionales.